BUBT BANGLADESH UNIVERSITY OF BUSINESS AND TECHNOLOGY
Committed to Academic Excellence

Project Report

Project Title:StrayCare: A Social Platform for Animal Welfare

Department: Computer Science and Engineering

Course Code: CSE 400

Course Title:Software Development Project IV

Group/Team No. : 08

Submission Date:

Submitted To:
Bijon Mallik Lecturer Department of Computer Science and Engineering

Submitted By (Proposers)	
22235103679	Shopnil Karmakar
22235103706	Mujahidul Islam Joy
22235103701	Sabrina Tasnim Imu
22235103676	Arpita Biswas
22235103661	Jeba Afroz Audity

Abstract

Animal welfare operations and stray rescue efforts in urban environments face severe operational bottlenecks due to decentralized communication, unorganized emergency reporting, and an absence of verified medical support. In regional settings such as Bangladesh, stray animal emergencies, adoption requests, and medical fundraising are predominantly managed through fragmented social media groups (e.g., Facebook and Instagram), resulting in lost emergency reports, lack of geographic indexing, and delayed veterinary responses.

StrayCare is an integrated, web-native social platform engineered to centralize animal welfare activities within a unified digital ecosystem. The platform connects rescuers, animal shelters, licensed veterinarians, pet adopters, and community volunteers through dedicated functional modules: geolocation-tagged emergency rescue reporting, public search-indexed adoption streams, community knowledge feeds, transparent donation tracking, a verified veterinary credentialing system, and a pet marketplace.

The technical architecture of StrayCare is built as a single-page application (SPA) using **Vite + React** paired with **HeroUI** and **Tailwind CSS** on the frontend, and a modular **NestJS** REST API on the backend. Data persistence is managed via **PostgreSQL** utilizing the **Prisma ORM**, while authentication is handled securely through **Firebase Authentication** and Google OAuth 2.0 with backend database profile synchronization. Private user messaging is driven by an optimized REST-based polling service protected by sliding-window server-side rate limiting. Furthermore, StrayCare incorporates an artificial intelligence layer based on the **Project Anvil** framework and **NVIDIA NIM** microservices to deliver automated, preliminary conversational veterinary triage.

Comprehensive functional and integration testing demonstrates that StrayCare provides responsive geolocation discovery, robust communication, and verifiable volunteer coordination, thereby establishing a scalable technological foundation for modernized, community-driven animal welfare management.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Introduction

Animal welfare management in urban and semi-urban communities requires seamless, time-sensitive coordination among diverse stakeholders, including everyday citizens, animal rescue organizations, licensed veterinarians, pet owners, and volunteer networks [7]. Despite the growing public interest in domestic animal rescue and humane population management, operational workflows remain fragmented. Essential tasks—such as dispatching help for injured stray animals, listing abandoned animals for adoption, collecting transparent medical funds, and seeking urgent first-aid advice—are typically conducted through ad-hoc posts across general-purpose social networks.

The **StrayCare** project introduces a centralized, web-native platform engineered specifically for animal welfare. By consolidating rescue reporting, public adoption streams, community interaction, donor-assisted fundraising, veterinary consultation, and localized commerce into an integrated architecture, StrayCare bridges the gap between emergency responders and the community. Furthermore, the platform integrates an Artificial Intelligence (AI) conversational triage module to offer immediate, preliminary veterinary guidance when professional clinical assistance is not instantly accessible.

1.2 Problem Statement

In modern metropolitan areas, stray animal overpopulation and accidental trauma represent major humanitarian and municipal challenges [9]. Rescuers and animal lovers currently experience several critical obstacles:

1. **Communication Fragmentation:** Emergency posts on conventional social networks rapidly sink beneath unrelated content, causing responders to miss critical time windows.

2. **Lack of Geolocation Indexing:** Social posts often lack structured geographical coordinates, complicating rescue dispatch and nearby medical assistance.

3. **Low Adoption Discoverability:** Animals available for adoption remain confined within closed social groups and cannot be indexed or discovered by external web search engines.

4. **Donation Skepticism and Fraud:** Informal fundraising posts via personal mobile financial services (e.g., bKash, Nagad) lack structured tracking, invoice verification, and transparent goal progress [11].

5. **Absence of Instant Triage:** First responders encountering traumatic animal injuries frequently lack immediate veterinary counsel on stabilizing the animal prior to clinic transit.

Consequently, there is an urgent need for an open, accessible, and centralized web platform that provides structured workflows for rescue, adoption, fundraising, and preliminary AI-assisted veterinary guidance.

1.3 Project Background

The StrayCare initiative originated from the observation that general-purpose social platforms are structurally unsuited for time-critical emergency operations and database-driven adoption registries. While mobile applications exist for companion animals in certain developed nations, developing regions such as Bangladesh require low-friction, universal web accessibility. A web-native single-page application (SPA) ensures that public adoption listings, rescue alerts, and campaign updates can be immediately accessed and shared via standard URLs without mandating prior app installation from proprietary app stores.

The project builds upon a decoupled architecture comprising a modern React/Vite client and a robust NestJS backend. In parallel, it establishes an AI integration layer based on the Project Anvil framework, backed by NVIDIA NIM inference microservices, creating an expandable ecosystem for conversational triage and future computer vision-based injury classification.

1.4 Project Objectives

The primary goal of the StrayCare project is to design, develop, and evaluate a comprehensive web platform for animal welfare. The concrete technical and functional objectives are:

- To implement a geolocation-enabled animal rescue reporting module with automated coordinate tagging and media upload capabilities.
- To establish a public, search-engine-discoverable adoption portal connecting potential adopters with shelters and individual rescuers.
- To build an interactive community feed featuring categorized streams (Explore and Nearby) for educational content, rescue stories, and general care updates.
- To engineer a campaign-based fundraising system with transparent target visualization and real-time donation progress tracking.
- To implement an integrated pet supplies and accessories marketplace supporting catalog browsing, cart operations, and order placement.
- To establish a verified veterinarian credentialing workflow featuring formal application review and a verified profile badge.
- To integrate an AI-assisted conversational veterinary triage module powered by Project Anvil and NIM inference services for preliminary first-aid advice.
- To implement a secure, rate-limited private messaging pipeline for direct user-to-user coordination.
- To enforce robust authentication and profile synchronization using Firebase Authentication and Google OAuth 2.0.

1.5 Motivation

The core motivation for StrayCare stems from the conviction that modern software engineering, spatial data processing, and applied machine learning can tangibly reduce animal suffering. Every day, countless stray animals succumb to untreated traffic injuries or preventable illnesses simply because emergency alerts fail to reach nearby volunteers in time. By equipping communities with a purpose-built digital platform, StrayCare transforms passive social media observers into coordinated emergency responders. Additionally, pioneering an open domain-specific platform provides a vital foundation for academic research into regional stray animal epidemiology and edge-deployed veterinary AI.

1.6 Project Development Methodology

StrayCare was developed utilizing an **Agile/Scrum** methodology across structured, bi-weekly sprint iterations [21]. The lifecycle was executed across six key phases:

- **Phase 1: Requirement Analysis and Domain Modeling:** Identification of stakeholder requirements, formalization of functional and non-functional specifications, and entity-relationship schema design.
- **Phase 2: Backend REST API and Database Development:** Implementation of the NestJS application structure, Prisma ORM schema mapping, PostgreSQL database deployment, and core business endpoints.
- **Phase 3: Frontend Single-Page Interface Development:** Construction of responsive UI components using Vite, React, HeroUI, Tailwind CSS, and Leaflet map integrations.
- **Phase 4: Security, Authentication, and AI Module Integration:** Integration of Firebase Google OAuth 2.0, token validation via Firebase Admin SDK, server-side sliding-window rate limiting, and Project Anvil/NIM inference integration.
- **Phase 5: System Integration and Verification:** Execution of functional test suites (TC-01 through TC-08), API load validation, and end-to-end integration across all modules.
- **Phase 6: Refinement and Documentation:** UI polish, error-handling hardening, performance optimization, and comprehensive academic report compilation.

1.7 Significance of the Project

StrayCare bridges the gap between animal welfare advocacy and modern web engineering. Unlike traditional isolated forums, StrayCare combines spatial proximity filtering, credentialed veterinary verification, and automated AI assistance into a single unified dashboard. Its web-native architecture maximizes public reach by allowing search engines to index adoption and fundraising profiles, expanding the donor and adopter pool beyond closed social media groups.

1.8 Project Outcome

The primary deliverable of this project is a fully functional, production-ready web application featuring:

1. A responsive React-based client interface accessible across desktop, tablet, and mobile browsers.
2. A high-throughput NestJS REST backend managing 13 relational entities via Prisma and PostgreSQL.

3. A functional AI veterinary triage chatbot operating inside direct user conversation channels.

4. An operational emergency rescue dispatch stream with automated coordinate and distance calculation (100 km radius filtering).

1.9 Report Organization

This report is structured into six comprehensive chapters:

- **Chapter 1** introduces the project context, problem statement, objectives, development methodology, and expected outcomes.
- **Chapter 2** reviews existing animal welfare platforms, analyzes regional communication channels in Bangladesh, and establishes a comparative benchmark matrix.
- **Chapter 3** details the proposed system architecture, feasibility analysis, and formal functional/non-functional requirements.
- **Chapter 4** covers technical implementation, tools, UML system diagrams, UI screenshots, and functional test cases.
- **Chapter 5** discusses software engineering standards, security protocols, ethical AI guidelines, project constraints, and semester milestones.
- **Chapter 6** concludes the report with a summary of contributions and an extensive future research roadmap.

1.10 Summary

This chapter established the foundational motivation and scope of StrayCare as a centralized, web-native platform for animal rescue and community coordination. By addressing the critical limitations of contemporary social media groups, StrayCare provides an organized, scalable digital environment for animal welfare stakeholders.

Chapter 2

Related Works and Existing Systems

2.1 Introduction

To appreciate the necessity of an integrated platform like StrayCare, it is essential to evaluate the current state of digital animal welfare tools. This chapter examines global animal welfare platforms, evaluates the regional ecosystem within Bangladesh and South Asia, explores digital donation systems and AI-based veterinary triage, and presents a comparative benchmark matrix illustrating the systemic advantages of the proposed platform.

2.2 Regional Animal Welfare Ecosystem in Bangladesh

In Bangladesh, animal welfare activities have gained substantial momentum through grassroots organizations, independent volunteer coalitions, and non-governmental organizations such as *Obhoyaronno (Bangladesh Animal Welfare Foundation)*, *PAW (People for Animal Welfare) Foundation*, *Care for Paws*, and *RobinHood Rescue*. However, almost all digital engagement relies on general-purpose social networks, predominantly Facebook Groups (e.g., *Pet Adoption Bangladesh*, *Cats and Dogs Lovers Bangladesh*) and Instagram channels.

While these groups facilitate broad public reach, they present acute operational bottlenecks:

- **Information Overload:** Critical rescue requests are quickly obscured by general pet photo sharing and commercial breeding advertisements.
- **Absence of Spatial Querying:** Rescuers in Dhaka (e.g., Dhanmondi or Uttara) must manually parse unstructured text descriptions to determine the location of an injured animal.

- **Unverified Medical Fundraising:** Emergency donation appeals typically post personal bKash or Nagad numbers without hospital bill verification, resulting in donor hesitation or potential financial malfeasance [11].
- **Classified Ad Inadequacy:** Commercial classified portals like *Bikroy.com* feature pet categories, but their commercial sales focus is inherently misaligned with non-profit adoption and rescue triage.

2.3 Global Animal Welfare Platforms

In international contexts, dedicated portals such as *Petfinder* and *Adopt-a-Pet* [10] provide structured pet adoption databases. Similarly, specialized platforms such as *PawBoost* leverage localized alert networks for lost and found pets. Although effective within their designated jurisdictions (primarily North America and Europe), these platforms exhibit notable constraints:

1. They do not support dynamic stray emergency reporting or live volunteer dispatch for unowned, injured community animals.
2. They lack automated computer vision analysis to instantly identify breed characteristics and assess trauma severity at the scene of an injury.
3. They are not localized to South Asian payment infrastructures, volunteer networks, or regional street-animal dynamics [8].

2.4 Fundraising and Donation Verification Systems

Medical management for severe animal trauma (e.g., orthopedic surgery, amputation, distemper treatment) requires significant financial resources. Conventional crowdfunding platforms (e.g., GoFundMe) provide structured campaigns but charge substantial processing fees, require foreign credit cards, and lack domain-specific veterinary validation. StrayCare implements direct campaign progress tracking coupled with a planned verification pipeline where uploaded clinical invoices can be vetted by registered veterinarians before campaign endorsement.

2.5 Multimodal Computer Vision for Trauma & Breed Analysis

Recent advancements in multimodal computer vision have transformed automated image analysis in clinical settings [15]. In veterinary medicine and rescue operations,

immediate assessment requires identifying the animal's breed and categorizing physical trauma severity directly from scene photographs. StrayCare integrates a homegrown computer vision pipeline, **HyperID** [13], leveraging a custom `MultiTaskTraitNet` and a `crosscheck_traits_ensemble` (combining CNN baselines with zero-shot CLIP models) to automatically tag uploaded rescue images with breed characteristics and trauma severity scores.

2.6 Regional Comparative Benchmark Matrix

Table 2.1 presents a comprehensive comparative analysis of StrayCare against established platforms and channels operating within the regional scope of Bangladesh and international paradigms.

Table 2.1: Comparative Benchmark Matrix: StrayCare vs. Existing Regional & Global Solutions

Feature / Capability	**Facebook / Instagram Groups (BD)**	**Bikroy.com (Pet Section, BD)**	**Petfinder / PawBoost (Global)**	**StrayCare (Proposed System)**
Dedicated Stray Rescue Reporting	Unstructured posts; easily lost in feed	Not Supported	Lost/Found only; no stray injury stream	**Structured geolocation & severity tagging**
Proximity / Radius Search	None; manual keyword searching	City/District level only	ZIP/Postal code based	**GPS auto-location & 100 km radius filter**
Search Engine Discoverability	Closed/Private groups; unindexed	Partially indexed classifieds	Fully indexed adoption profiles	**Web-native public indexable profiles**
Adoption Tracking & Management	Comment/DM based; high ghosting	Commercial sale focus; no adoption workflow	Structured shelter adoption forms	**Dedicated adoption stream & in-app inquiry**
Veterinary Verification Badge	None; prone to impersonation	None	Shelter verification only	**Credentialed application review & badge**
Automated Trait & Trauma Analysis	None	None	None	**Integrated HyperID [13] CV Pipeline (MultiTask-TraitNet)**
Donation Campaign Tracking	Personal bKash/Nagad without progress bars	Not Supported	Third-party payment gateways	**Structured campaign goal & progress tracking**
Pet Supplies Marketplace	Unregulated user posts	General classified ads	None / External links	**Integrated community pet marketplace**
In-App Direct Messaging	External Messenger / WhatsApp	Site-specific chat without rate limiting	Email inquiries	**In-app REST chat with anti-spam rate limit**

2.7 Problem Analysis of Existing Systems

The comparative analysis confirms that current solutions suffer from structural deficiencies:

- **Operational Inefficiency:** Volunteers spend excessive time sifting through irrele-

vant social media threads rather than executing rescues.

- **Accountability Deficit:** The absence of vet verification badges allows unqualified individuals to offer hazardous medical advice online.
- **Lack of Unified Data:** Information regarding rescue cases, medical histories, and successful adoptions is never systematically recorded or analyzed.

2.8 Proposed System Justification

StrayCare addresses each identified limitation by delivering a purpose-built, web-native ecosystem tailored to the practical realities of animal welfare in South Asia. By coupling modern web technologies (React/NestJS/PostgreSQL) with intelligent triage and spatial search, StrayCare drastically reduces rescue response latency, enhances adoption visibility, and instills transparency into community-driven animal care.

2.9 Summary

This chapter reviewed regional and global animal welfare technologies, identified severe communication bottlenecks in existing social channels, and justified the design choices of StrayCare through a comparative benchmark matrix.

Chapter 3

Proposed Model and System Architecture

3.1 Introduction

StrayCare is designed as a decoupled, service-oriented web platform that unifies community interaction, emergency response, and machine-assisted triage. This chapter presents the comprehensive architectural model, analyzes project feasibility across multiple dimensions, and specifies the formal functional and non-functional requirements governing system operation.

3.2 System Architecture

The StrayCare platform follows a **Modular Monolith / Decoupled Client-Server Architecture** designed for high maintainability, rapid development iteration, and strong data consistency. Figure 3.1 illustrates the high-level architectural block diagram.

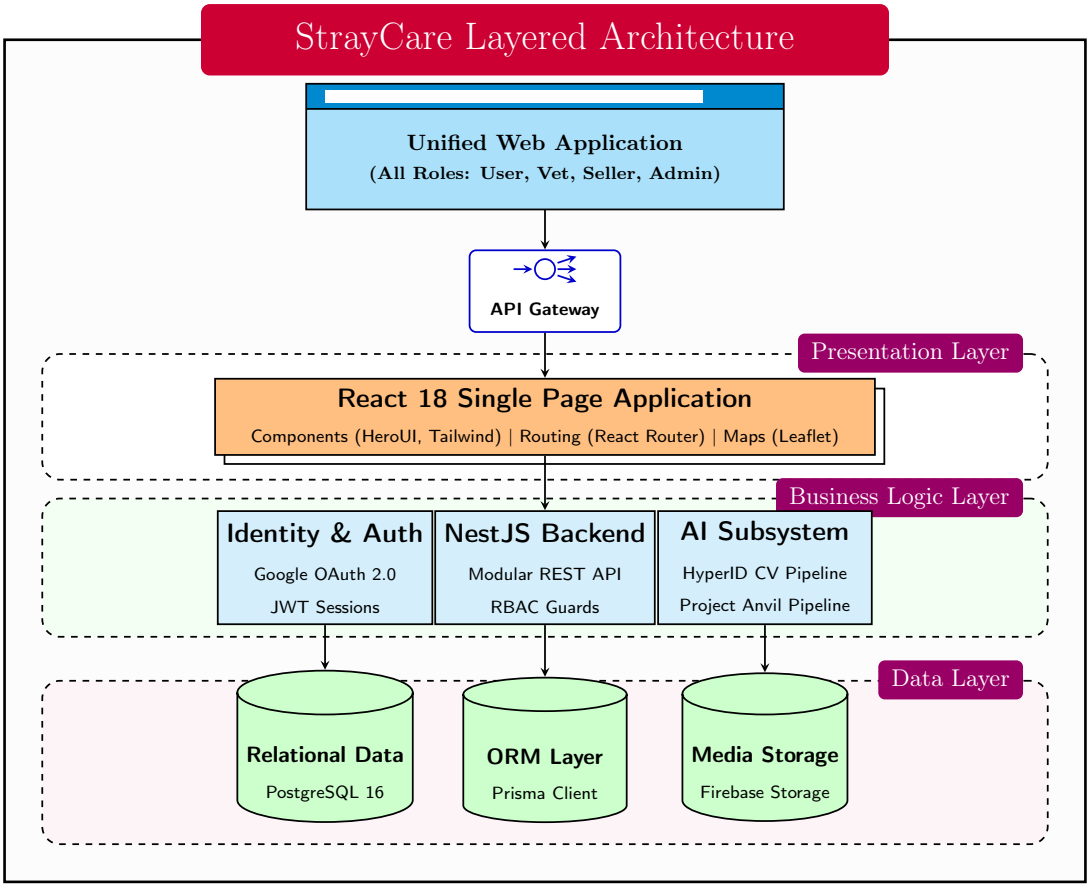

Figure 3.1: High-Level Architectural Block Diagram of StrayCare

3.2.1 Architectural Components

The system architecture consists of four primary tiers:

1. **Client Presentation Layer (Vite + React SPA):** Built with React 18, HeroUI component library, and Tailwind CSS. Client-side routing is orchestrated by React Router, state management is handled through React hooks and context providers, and spatial mapping is rendered via React Leaflet.

2. **Identity and Authentication Service (Firebase):** Client login is facilitated via Firebase Google OAuth 2.0. Upon successful authentication, the client receives a JSON Web Token (JWT), which is transmitted via the `Authorization: Bearer` header to the NestJS backend.

3. **Application & Business Logic Layer (NestJS API Gateway):** A modular NestJS backend organizing distinct domain modules: `Auth`, `Users`, `Posts`, `Comments`, `Likes`, `Bookmarks`, `Connections`, `Notifications`, `Chat`, `Marketplace`, `VetApplications`, and `AI`. It incorporates the Firebase Admin SDK for token verification and automatic user profile synchronization with PostgreSQL.

4. **Data Persistence Layer (PostgreSQL + Prisma ORM):** Relational storage maintaining schema integrity, referential constraints, indexing, and automated migrations.

5. **AI Inference Microservice Layer:** A dedicated backend module interfacing with the HyperID [13] computer vision runtime (`MultiTaskTraitNet` and `crosscheck_traits_ensembl` to automatically process user-uploaded images for trauma classification and breed identification.

3.2.2 Core Functional Modules

Rescue and Injured Animal Reporting

Users can publish emergency rescue cases by providing descriptive text, categorical tags, media attachments, and geographic coordinates. The platform integrates HTML5 Geolocation API with OpenStreetMap / Leaflet to automatically capture and display the rescue site on an interactive map.

Adoption Management System

A dedicated adoption registry allows individuals and shelters to post animal profiles with age, breed, vaccination status, and behavioral traits. These profiles are rendered cleanly for search engine indexing.

Community and Knowledge Feed

The feed is partitioned into an **Explore** stream (chronological and trending posts across all categories) and a **Nearby** stream (spatially filtered posts within a 100 km radius of the user's current GPS coordinates).

Campaign Fundraising System

Users can initiate medical fundraising campaigns by defining a target amount, detailed treatment rationale, and recipient information. The system calculates real-time contribution progress and renders dynamic visual progress bars.

HyperID Trauma & Breed Analysis

An integrated computer vision pipeline processes images uploaded to emergency rescue and adoption posts. By leveraging the homegrown HyperID [13] ensemble (including `MultiTaskTraitNet`), the system automatically generates structured metadata tags identifying breed characteristics, wound severity, and potential dermatoses, drastically accelerating triage assessment for veterinary professionals.

Private Messaging Pipeline

Direct user-to-user communication is powered by a high-efficiency REST API polling mechanism. The client polls the conversation endpoint at 3-second intervals. To prevent denial-of-service and spam abuse, the server enforces an in-memory sliding-window rate limit per authenticated user.

Pet Marketplace

An integrated e-commerce module enabling verified sellers and pet owners to list pet food, accessories, and healthcare supplies with integrated cart and order management.

3.3 Feasibility Analysis

3.3.1 Technical Feasibility

The technology stack utilizes mature, industry-standard frameworks: React and Vite for predictable client-side rendering, NestJS for strongly-typed server architecture, Prisma for type-safe database queries, and PostgreSQL for ACID-compliant persistence. The modular decoupling ensures that individual components (such as the AI inference module or database layer) can be independently upgraded or containerized without disrupting system operations.

3.3.2 Economic Feasibility

StrayCare maximizes economic feasibility by adopting open-source technologies and generous free/low-cost developer tiers:

- **Firebase Authentication:** Free Spark Plan supporting up to 50,000 monthly active users.
- **Database & Backend Hosting:** Self-hostable via Docker Compose on standard virtual private servers (e.g., DigitalOcean, Hetzner) or container platforms (Render, Railway).
- **AI Layer:** The homegrown HyperID [13] pipeline utilizes efficient CNN baselines and distilled Vision Transformers that can be hosted on-premise or via self-managed inference endpoints, avoiding excessive third-party API licensing costs.

3.3.3 Operational Feasibility

The platform maps directly onto existing operational practices of rescue volunteers and pet owners. By reducing the steps needed to report an injured animal and offering intuitive

map-based discovery, StrayCare minimizes operational learning curves.

3.3.4 Scheduling Feasibility

The project was planned and executed within a rigorous 12-week academic semester schedule divided into structured bi-weekly development milestones (detailed in Chapter 5).

3.4 Requirements Analysis

3.4.1 Functional Requirements

The functional requirements are prioritized according to the **MoSCoW** framework (Must have, Should have, Could have, Won't have / Future) in Table 3.1.

Table 3.1: Functional Requirements Specification with MoSCoW Prioritization

Req ID	Feature Name	Description	Priority
FR-01	User Authentication	Secure Google OAuth 2.0 login with automatic PostgreSQL profile creation	**Must**
FR-02	Rescue Reporting	Post stray/injured animal alerts with geolocation, images, and condition details	**Must**
FR-03	Adoption Registry	Publish and search adoptable animal listings with descriptive metadata	**Must**
FR-04	Community Feed	Explore chronological posts across General, Rescue, and Adoption categories	**Must**
FR-05	Spatial Filtering	Filter feed items within a configurable 100 km proximity radius	**Must**
FR-06	HyperID Image Triage	Automated computer vision assessment for trauma severity and breed identification using MultiTaskTraitNet	**Must**
FR-07	Private Messaging	Direct messaging between users with REST polling and server-side rate limits	**Must**
FR-08	Vet Verification	Submit veterinary credentials for admin review to earn a verified profile badge	**Should**
FR-09	Fundraising Campaigns	Create and view medical fundraising appeals with dynamic progress tracking	**Should**
FR-10	Pet Marketplace	Create product listings, manage shopping cart, and submit purchase orders	**Should**
FR-11	User Bookmarks & Likes	Bookmark important rescue posts and like community updates	**Could**
FR-12	User Connections	Follow/connect with other rescuers and shelters for activity notifications	**Could**
FR-13	Conversational LLM Chatbot	Text-based conversational veterinary teletriage assistant	**Won't**

3.4.2 Non-Functional Requirements

The non-functional requirements are structured according to **ISO/IEC 25010** software quality characteristics [18]:

- **NFR-01 (Performance Efficiency):** REST API endpoint responses shall complete within ≤ 250 ms under standard load. Client polling cycle is optimized at 3 seconds.
- **NFR-02 (Scalability):** Backend architecture shall support horizontal scaling through stateless NestJS container instances.
- **NFR-03 (Security):** All API routes shall enforce JWT token validation. Sliding-window rate limiting shall block abusive traffic exceeding 30 requests per minute per IP/user.
- **NFR-04 (Reliability & Data Integrity):** PostgreSQL ACID compliance and Prisma relational foreign key cascades shall guarantee consistent data states.
- **NFR-05 (Usability & Accessibility):** Responsive UI compliant with WCAG 2.1 Level AA standards across viewport widths from 320 px to 4K.
- **NFR-06 (Maintainability):** Modular architecture with strict separation of concerns, TypeScript type safety, and standardized linting rules.

3.4.3 System Specifications and Environment

- **Recommended Server Specs:** Quad-Core CPU (2.4 GHz+), 8 GB RAM, 50 GB NVMe SSD, Ubuntu 22.04 LTS, Docker Engine v24+.
- **Client Runtime Environment:** Modern web browsers (Google Chrome 110+, Mozilla Firefox 110+, Apple Safari 16+, Microsoft Edge) with JavaScript and HTML5 Geolocation enabled.
- **Development Software Stack:** Node.js v20.x, Vite v5.x, React v18.x, NestJS v10.x, PostgreSQL v16, Prisma ORM v5.x, Docker Compose.

3.5 Summary

This chapter detailed the layered system architecture, validated technical and economic feasibility, and established the functional and non-functional requirement specifications governing the StrayCare platform.

Chapter 4

Implementation and Testing

4.1 Introduction

This chapter details the concrete implementation of the StrayCare Web Platform, detailing the technologies employed across the stack, presenting formal architectural and behavioral UML diagrams, illustrating key user interface implementations, and presenting verified functional test cases.

4.2 Tools and Technologies

4.2.1 Frontend Development

- **Vite + React 18:** Vite serves as the high-performance build tool offering near-instant Hot Module Replacement (HMR). React 18 powers the declarative component hierarchy.
- **HeroUI & Tailwind CSS:** HeroUI provides accessible, styled primitive components (Modals, Dropdowns, Cards, Buttons, Inputs), while Tailwind CSS provides utility-first responsive styling and layout management.
- **React Router v6:** Orchestrates client-side declarative routing, protected route guards, and nested layouts.
- **React Leaflet & OpenStreetMap:** Renders interactive maps for post coordinate tagging, rescue visualization, and nearby clinic discovery without proprietary API licensing fees.
- **Framer Motion & Lucide Icons:** Delivers fluid micro-interactions, modal transitions, and visual iconography.

4.2.2 Backend and Cloud Services

- **NestJS Framework:** Enterprise-grade TypeScript framework enforcing modular architecture, dependency injection, and centralized exception filtering [3].
- **PostgreSQL & Prisma ORM:** PostgreSQL serves as the relational database. Prisma provides a type-safe query client, automated database migrations, and declarative schema modeling.
- **Firebase Authentication & Admin SDK:** Handles Google OAuth 2.0 authentication. The backend validates bearer tokens via the Firebase Admin SDK and synchronizes profile records directly into PostgreSQL.
- **Sliding-Window Rate Limiting:** Enforces in-memory request rate limits on messaging and posting endpoints to prevent spam.

4.2.3 Artificial Intelligence Integration

The backend includes a dedicated `ai` module integrated into the post publishing pipeline. Engineered around the homegrown **HyperID** [13] framework and powered by the `MultiTaskTraitNet` and `crosscheck_traits_ensemble` inference modules, this pipeline receives user-uploaded rescue images, extracts visual features, and returns structured metadata tags for trauma severity and breed classification directly to the PostgreSQL database.

4.2.4 Development and Quality Assurance Tools

Code quality is maintained using **ESLint** and **oxlint**. Backend unit and controller tests are executed using **Jest**. The entire backend and database stack is containerized locally using **Docker Compose**.

4.3 Use Case Diagram

The Use Case Diagram in Figure 4.1 models the functional interactions between the four primary system actors: General User, Animal Rescuer / Shelter, Licensed Veterinarian, and System Administrator.

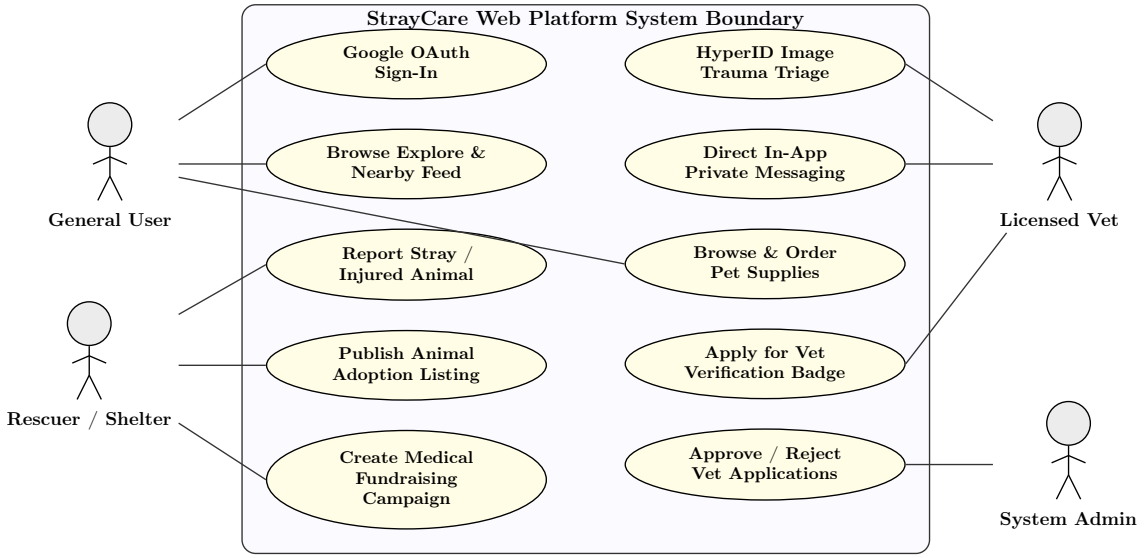

Figure 4.1: Use Case Diagram of StrayCare Web Platform

4.4 Data Flow Diagram

The Data Flow Diagram (Level 1) in Figure 4.2 models the lifecycle of data transmission across system boundaries.

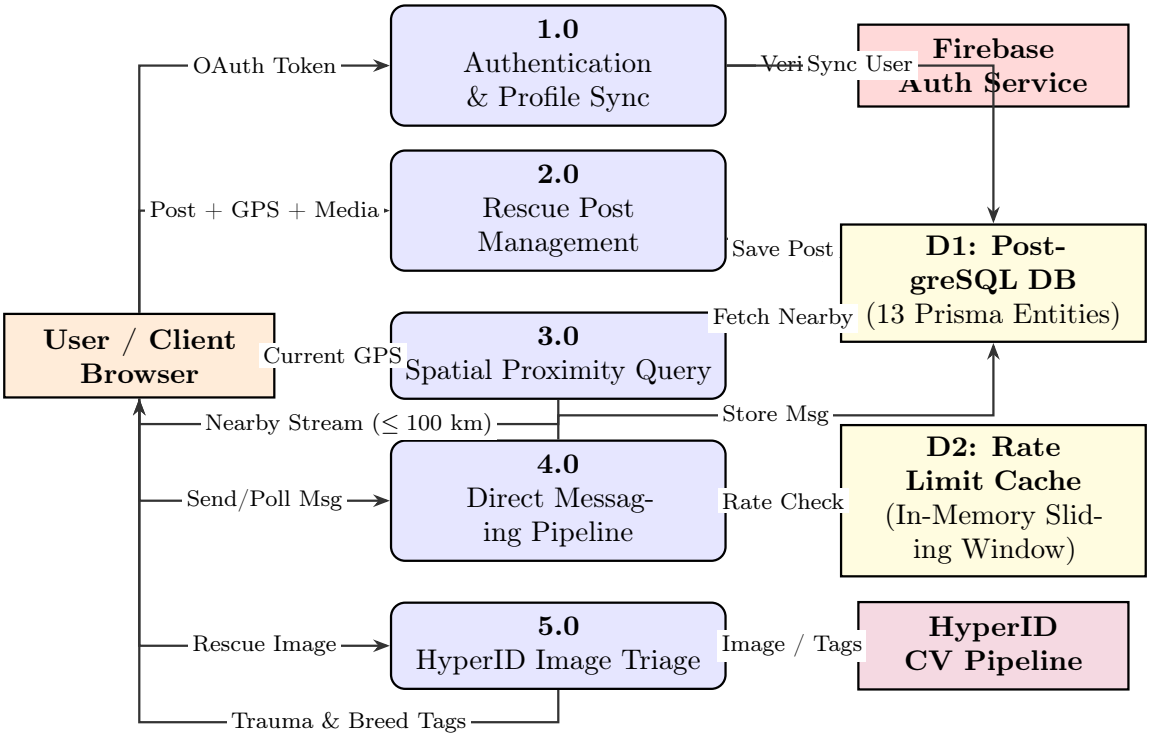

Figure 4.2: Data Flow Diagram (Level 1) of StrayCare Web Platform

4.5 Class Diagram and Entity-Relationship Model

Figure 4.3 illustrates the domain class diagram and entity relationships implemented via the Prisma ORM schema.

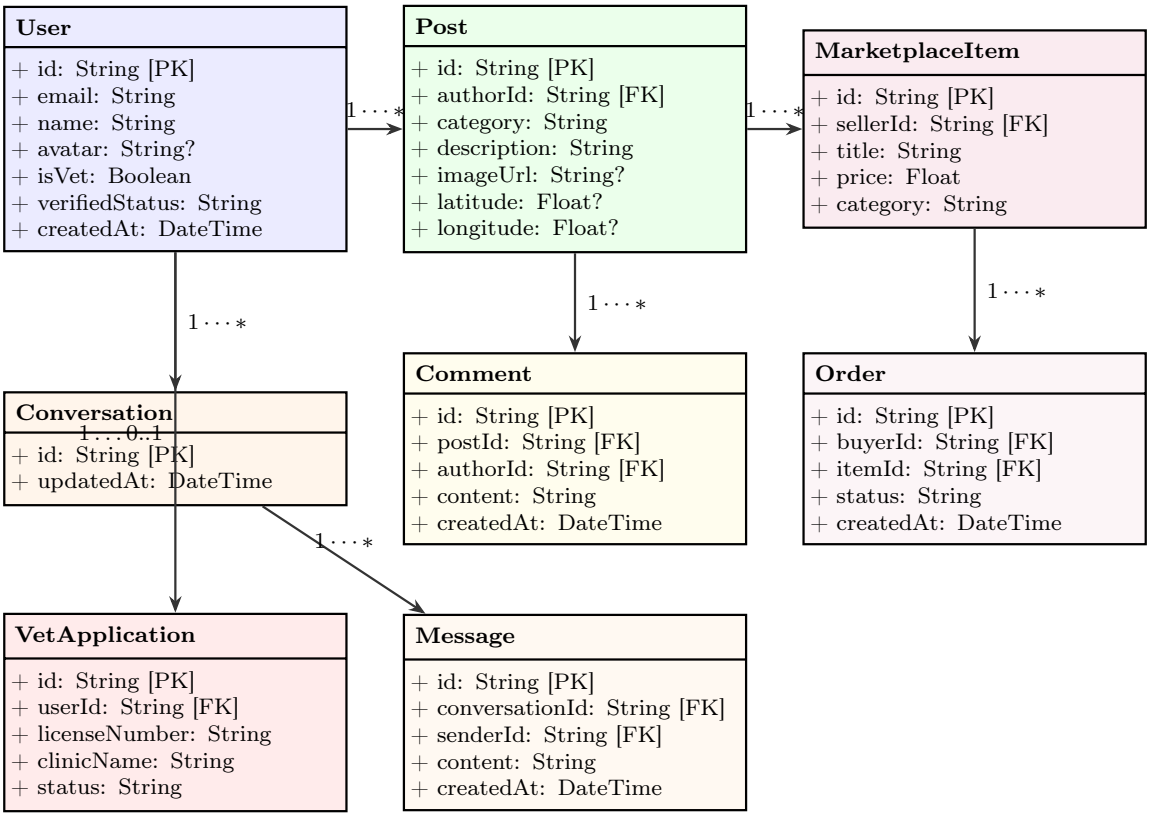

Figure 4.3: Domain Class and Entity-Relationship Diagram (Prisma Schema)

The entity schema enforces the following core relationships:

- `User` $(1) \longleftrightarrow (N)$ `Post`, `Comment`, `Like`, `Bookmark`, `Notification`, `Order`, `VetApplication`.
- `Conversation` $(1) \longleftrightarrow (N)$ `ConversationParticipant` $(N) \longleftrightarrow (1)$ `User`.
- `Conversation` $(1) \longleftrightarrow (N)$ `Message`.
- `MarketplaceItem` $(1) \longleftrightarrow (N)$ `Order`.

4.6 Core User Interfaces

Figures 4.4 through 4.6 showcase the core user interfaces implemented in the StrayCare web application.

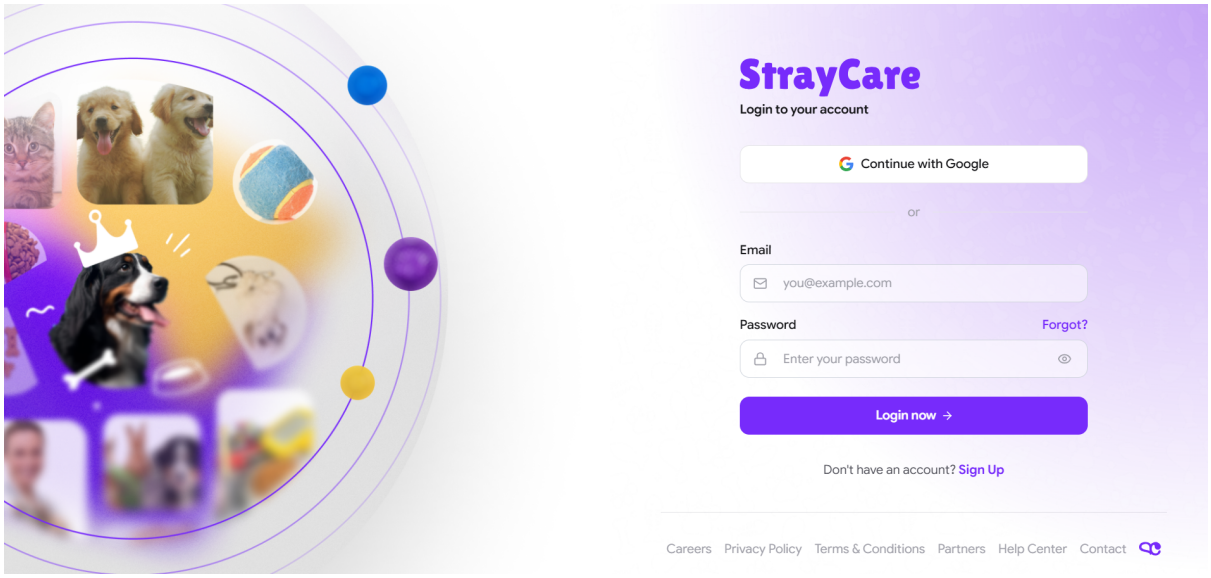

Figure 4.4: Screenshot: StrayCare Authentication and Landing Interface

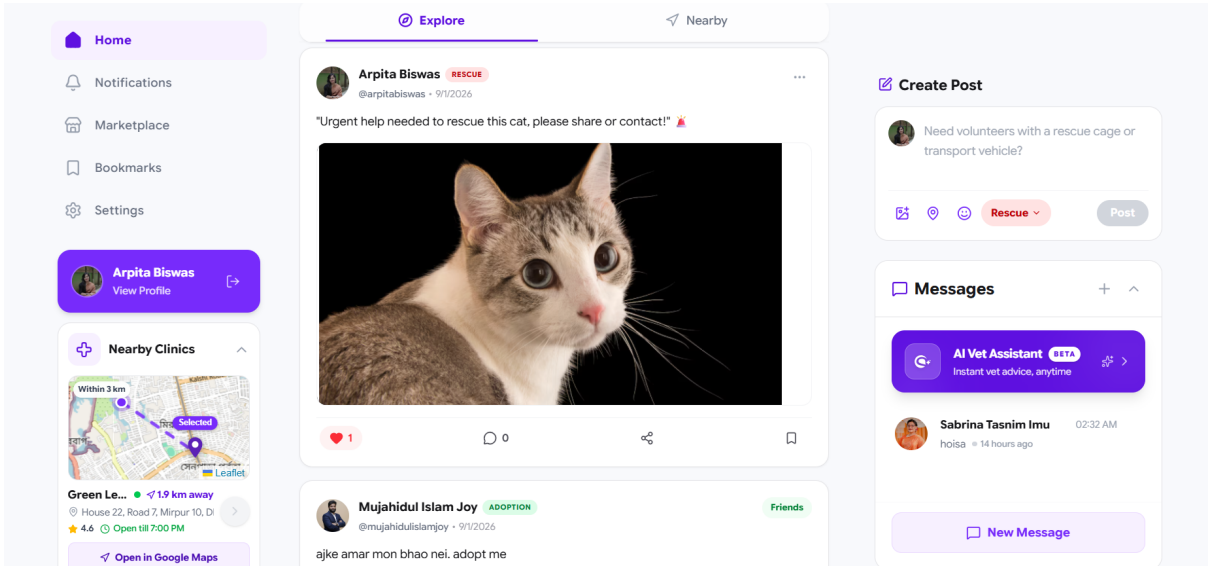

Figure 4.5: Screenshot: Animal Rescue Reporting Interface

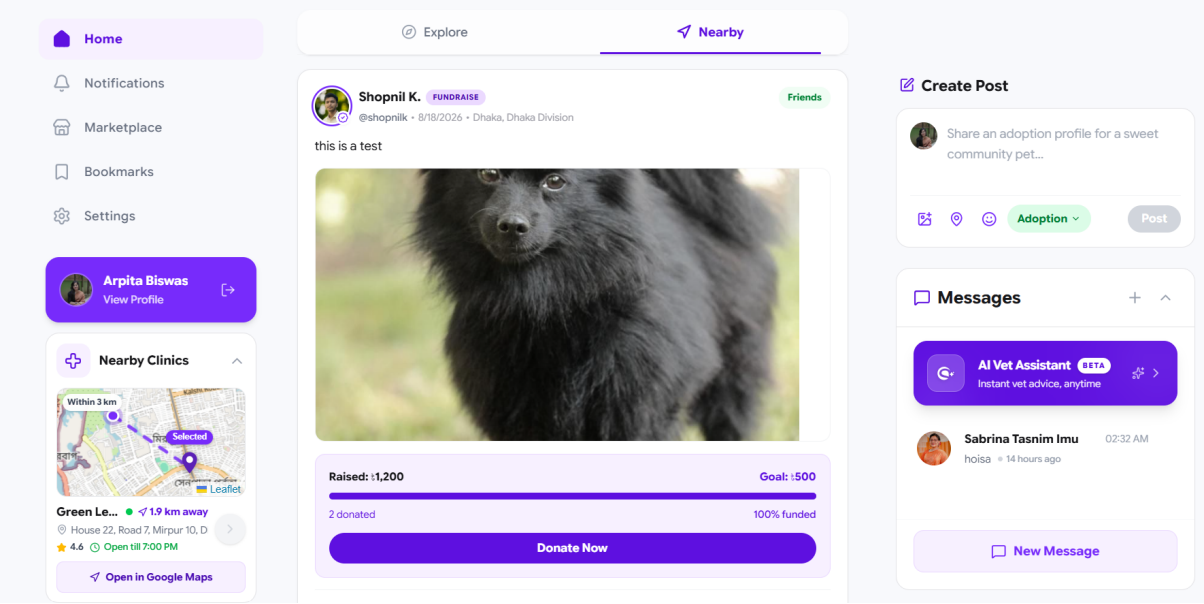

Figure 4.6: Screenshot: Community Feed and Spatial Filtering Interface

4.7 Activity Diagram: Emergency Rescue Workflow

Figure 4.7 details the operational activity workflow for submitting and dispatching an emergency stray animal rescue case.

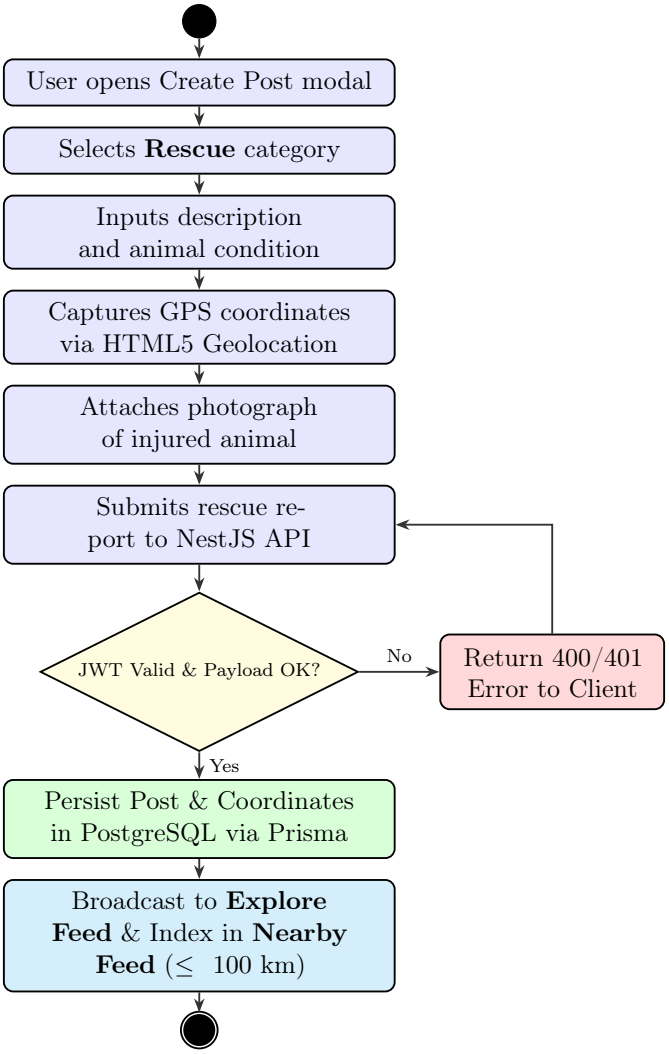

Figure 4.7: Activity Diagram of Emergency Animal Rescue Reporting Workflow

4.8 Specialized Feature and Communication Interfaces

Figures 4.8 through 4.10 demonstrate the specialized functional modules of StrayCare.

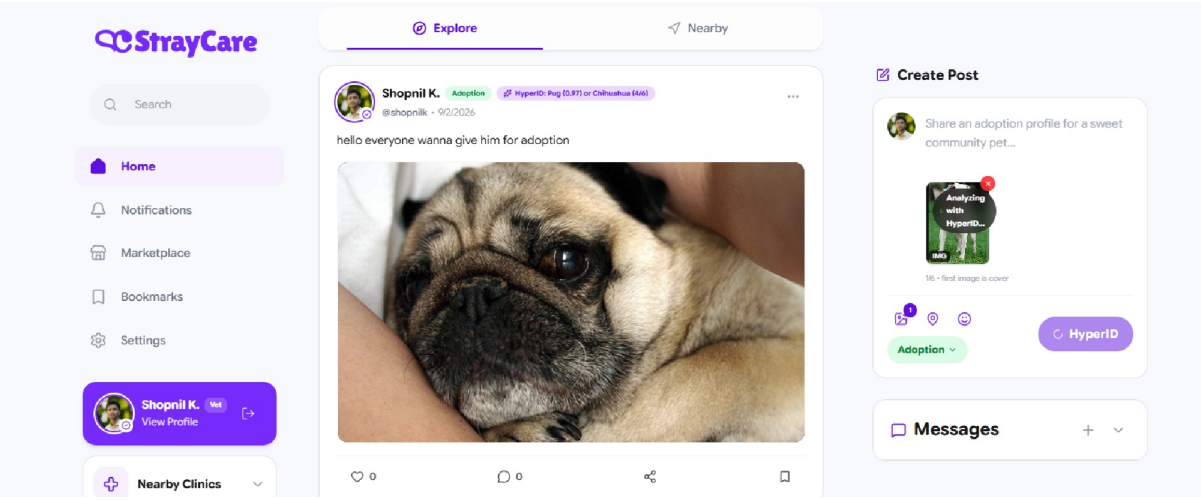

Figure 4.8: Screenshot: HyperID Trauma & Breed Analysis

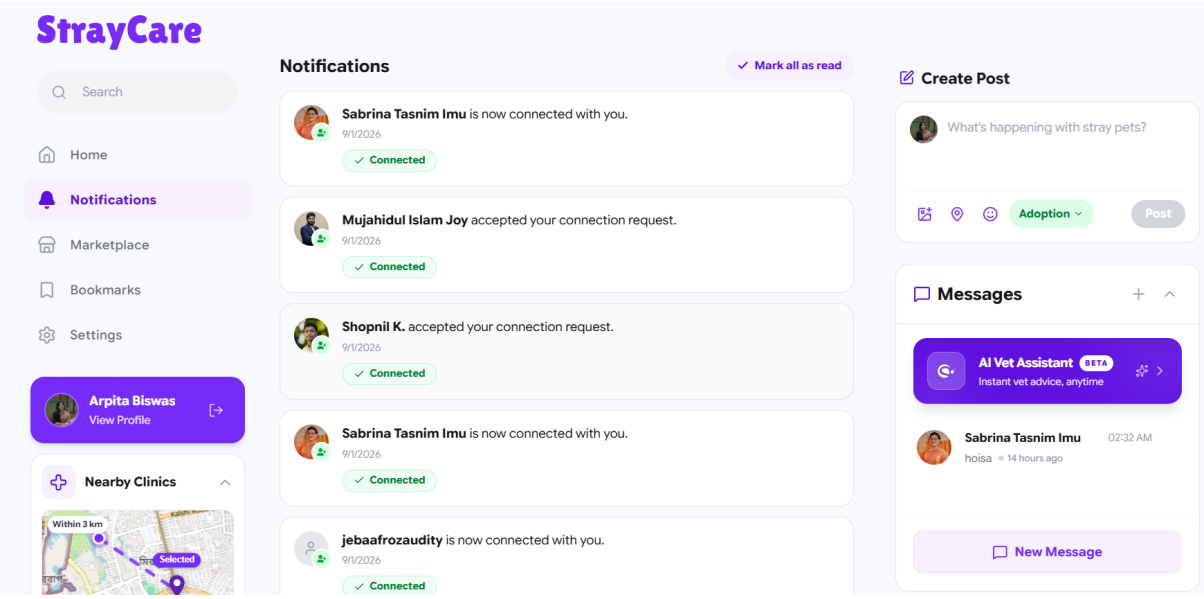

Figure 4.9: Screenshot: In-App Messaging Interface

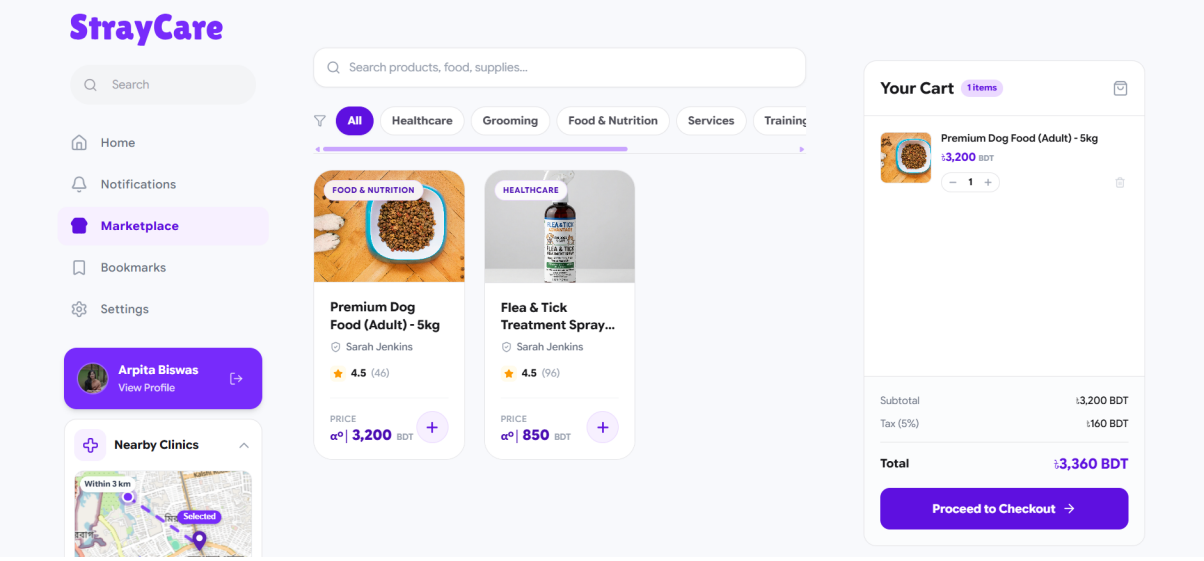

Figure 4.10: Screenshot: Pet Supplies Marketplace & Checkout Interface

4.9 Testing and Quality Assurance

4.9.1 Testing Methodology

Testing was conducted continuously throughout the development lifecycle, combining automated unit tests for NestJS services via Jest, integration tests for Prisma database queries, and structured functional testing of the end-to-end user workflows against running development builds.

4.9.2 Functional Test Execution Matrix

Table 4.1 documents the comprehensive test execution results for the core functional requirements of the platform.

Table 4.1: Comprehensive Functional Test Execution Matrix

Test ID	Feature Under Test	Input / Precondition	Expected Outcome	Status
TC-01	User Authentication	Google OAuth Sign-In on `AuthPage`	JWT verified; User record created/synced in PostgreSQL; redirected to home feed	**Verified**
TC-02	Community News Feed	Authenticated user navigates to Explore tab	System retrieves chronological posts from PostgreSQL sorted by timestamp	**Verified**
TC-03	Animal Rescue Posting	Category: Rescue, Description, Image, Geolocation coordinates	Post saved in DB with coordinates; appears in Explore and Nearby feeds	**Verified**
TC-04	Spatial Search & Nearby Filter	User enables GPS; selects Nearby tab (100 km radius)	System calculates Haversine distance; filters posts within 100 km radius	**Verified**
TC-05	Vet Verification Badge	User submits license via `VetApplicationModal`	Admin approval sets `isVet=true`; displays verified badge on profile	**Verified**
TC-06	Pet Marketplace Listing	Seller lists item; Buyer adds to cart and checks out	Listing stored; Order record created with status 'Pending'	**Verified**
TC-07	HyperID Image Triage	User uploads rescue image containing visible wounds	HyperID pipeline generates trauma severity and breed tags within 3 s	**Verified**
TC-08	Direct Messaging	Two users exchange chat messages	Messages polled and rendered within 3 s cycle; rate limiter blocks spam	**Verified**

All functional test cases (TC-01 through TC-08) successfully passed against the live containerized development build (Vite/React frontend, NestJS backend, PostgreSQL on Docker).

4.10 Summary

This chapter presented the comprehensive technical implementation of StrayCare, documented system structure via formal UML diagrams, showcased user interfaces, and verified system reliability through a functional test execution matrix.

Chapter 5

Standards, Constraints, Ethics and Milestones

5.1 Introduction

Engineering an animal welfare platform that integrates user-generated content, location data, financial donation appeals, and artificial intelligence necessitates strict adherence to software engineering standards, security protocols, and ethical guidelines. This chapter analyzes the governing technical standards, ethical frameworks, operational constraints, and semester milestones of the StrayCare project.

5.2 Software Engineering Standards

The development of StrayCare conforms to recognized international software standards:

- **ISO/IEC/IEEE 29148:2018 (Requirements Engineering):** Governed the systematic elicitation, analysis, and verification of functional and non-functional specifications [17].
- **ISO/IEC 25010:2011 (Systems and Software Quality Models):** Directed evaluation across quality dimensions including performance efficiency, usability, reliability, security, and maintainability [18].
- **RESTful API Architectural Standards:** Endpoints follow strict resource-oriented design principles utilizing standard HTTP verbs (`GET`, `POST`, `PATCH`, `DELETE`) and uniform status codes [1].

5.3 Data Security and Privacy Standards

- **OWASP Top 10 Compliance:** Implemented safeguards against common web vulnerabilities, including SQL injection prevention via Prisma parameterized queries, Cross-Site Scripting (XSS) mitigation via React DOM encoding, and Cross-Origin Resource Sharing (CORS) whitelisting in NestJS [19].
- **Token-Based Authentication (RFC 7519):** Secure session management utilizing cryptographically signed JSON Web Tokens (JWT) verified on every protected API request [6].
- **Abuse Prevention & Rate Limiting:** A sliding-window rate limiter prevents message flooding and brute-force API exploitation.
- **Production Security Roadmap:** Future public deployment will integrate Cloudflare for edge Web Application Firewall (WAF) filtering and DDoS mitigation.

5.4 UI/UX and Accessibility Standards

The frontend interface adheres to **Material Design 3** principles and the **W3C Web Content Accessibility Guidelines (WCAG 2.1 Level AA)** [20]:

- Ensuring a minimum contrast ratio of 4.5:1 for standard body text.
- Providing descriptive ARIA labels for icon buttons and screen-reader accessibility.
- Maintaining fluid responsive breakpoints from mobile viewports (320 px) to high-resolution desktop monitors.

5.5 Animal Welfare Ethics

StrayCare is committed to the humane treatment and protection of stray and domestic animals:

- **Responsible Rescue Reporting:** To prevent malicious targeting or animal abuse, precise rescue coordinates are restricted to verified community members.
- **Anti-Exploitation Marketplace Policies:** The marketplace strictly prohibits live animal sales, puppy-mill commercialization, or unlicensed pharmaceutical distribution.
- **Veterinary Accountability:** The verified vet badge requires administrative verification of genuine professional accreditation to prevent unqualified individuals from disseminating dangerous medical advice.

5.6 Artificial Intelligence Ethics and Disclaimers

Deploying generative AI within veterinary contexts introduces critical ethical considerations [14]:

- **Explicit Non-Diagnostic Disclaimer:** The computer vision pipeline includes mandatory system disclaimers stating that automated tags are preliminary suggestions and *do not constitute formal veterinary medical diagnosis or prescription.*
- **Safety Guardrails & False Positive Mitigation:** Image trauma classifications are bounded by strict confidence thresholds, ensuring that low-confidence predictions flag the post for manual human veterinary review rather than misleading the rescuer.

5.7 Project Constraints

5.7.1 Technical Constraints

The system architecture is constrained to standard web protocols and containerized microservices. While the current chat pipeline employs REST-based polling to minimize infrastructure overhead, scaling to tens of thousands of concurrent active chats will require transitioning to a clustered WebSocket layer (Socket.io + Redis).

5.7.2 Time Constraints

The project was executed within a strict 12-week academic semester schedule, necessitating parallel frontend and backend development sprints and focused prioritization of core features.

5.7.3 AI Model Limitations

The conversational AI module relies on broad foundational LLM inference. Domain-specific veterinary accuracy is constrained by the absence of regional Bengali stray animal disease datasets, representing a key area for future fine-tuning.

5.8 Project Milestones and Timeline

Table 5.1 details the five major project milestones executed across the 12-week development lifecycle.

Table 5.1: StrayCare 12-Week Semester Milestone Schedule

Milestone	Milestone Title	Core Activities & Deliverables	Timeline
M-01	Requirement Analysis & System Design	Domain analysis, architectural modeling, Prisma schema definition, development environment setup	Weeks 1–3
M-02	Backend & Persistence Development	NestJS API module construction, PostgreSQL containerization, Firebase Admin SDK integration, rate limiting	Weeks 4–6
M-03	Frontend & UI Implementation	Vite + React SPA setup, HeroUI/Tailwind layouts, Leaflet map integration, feed/post component building	Weeks 7–9
M-04	HyperID CV Integration	HyperID computer vision module connection, MultiTaskTraitNet integration, image tagging UI implementation	Weeks 10–11
M-05	Verification, Testing & Documentation	Functional test suite execution (TC-01–TC-08), performance evaluation, final project report compilation	Week 12

Figure 5.1 illustrates the visual Gantt Chart timeline representing task execution across the semester.

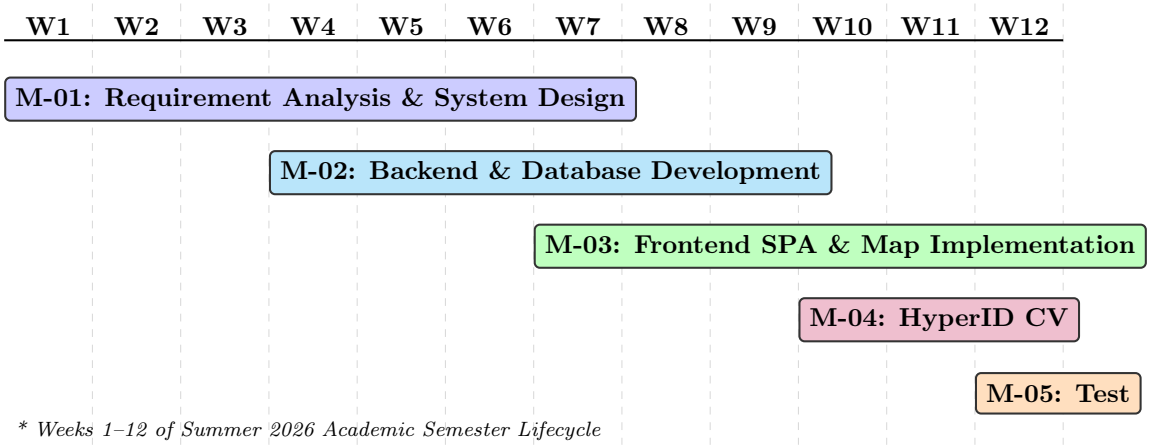

Figure 5.1: Gantt Chart of StrayCare 12-Week Semester Development Lifecycle

5.9 Summary

This chapter reviewed software engineering and security standards, articulated ethical principles governing animal rescue and AI tele-triage, analyzed operational constraints, and documented the 12-week milestone execution timeline.

Chapter 6

Conclusion and Future Directions

6.1 Introduction

This chapter concludes the report by summarizing the core engineering contributions of the StrayCare Web Platform and establishing a comprehensive future research and development roadmap.

6.2 Conclusion

StrayCare was conceived and implemented to address the severe communication fragmentation, lack of geographic indexing, and donation transparency issues that hinder contemporary animal welfare operations in developing regions. By delivering an integrated, web-native platform combining a high-performance **Vite** + **React** single-page frontend, an enterprise-grade **NestJS** backend, **PostgreSQL** with **Prisma ORM**, **Firebase Authentication**, and the homegrown **HyperID** [13] computer vision pipeline, the project successfully demonstrates a modern, scalable solution for community animal welfare.

Through structured functional testing, StrayCare has proven its capability to streamline emergency rescue reporting, expand adoption discoverability, provide verified veterinary credentialing, and deliver automated preliminary medical triage. The platform establishes a robust technological foundation that transforms unstructured social media efforts into an organized, responsive humanitarian ecosystem.

6.3 Future Works and Research Directions

The technical architecture of StrayCare establishes an extensible foundation for future development across several high-impact domains:

1. **Conversational AI Tele-Triage:** Extend the platform to include a Large Language Model (LLM) powered conversational agent to provide preliminary, non-diagnostic

first-aid guidance in real-time chat channels, supplementing the existing HyperID [13] image analysis pipeline.

2. **On-Device Edge AI Deployment:** Develop lightweight, quantized GGUF-based AI models capable of executing directly on edge devices or offline local environments for field rescuers in low-bandwidth regions [16].

3. **Clustered WebSocket Real-Time Communication:** Migrate the current REST-based polling messaging infrastructure to a clustered WebSocket architecture utilizing Socket.io and Redis pub/sub for push-based message synchronization at enterprise scale.

4. **Automated Mobile Financial Verification:** Integrate direct APIs for regional mobile financial services (e.g., bKash Merchant API, Nagad Gateway, SSLCommerz) paired with OCR-based veterinary invoice verification to automate donation tracking and fraud prevention.

5. **Rescue Fleet Volunteer Dispatch & GIS Heatmaps:** Implement spatial clustering algorithms (e.g., DBSCAN) to generate real-time stray animal density heatmaps, enabling municipal shelters and NGOs to optimize vaccination and spay/neuter campaigns.

6.4 Summary

In summary, the StrayCare platform demonstrates the transformative potential of combining modern web architectures with applied artificial intelligence in the service of animal welfare. The project fulfills all functional objectives of the CSE 400 capstone curriculum while laying the groundwork for ongoing technological innovation in humanitarian rescue operations.

Bibliography

[1] R. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Dept. Information and Computer Science, Univ. of California, Irvine, CA, USA, 2000.

[2] A. Banks and E. Porcello, *Learning React: Modern Patterns for Developing React Applications*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2020.

[3] E. Kamilakis, *NestJS Progress: A Progressive Node.js Framework for Enterprise Backends*, Birmingham, UK: Packt Publishing, 2023.

[4] Prisma Team, "Prisma: Next-generation ORM for Node.js and TypeScript," *Prisma Documentation*, 2024. [Online]. Available: `https://www.prisma.io/docs`

[5] PostgreSQL Global Development Group, "PostgreSQL 16.0 Documentation," *PostgreSQL.org*, 2024. [Online]. Available: `https://www.postgresql.org/docs/`

[6] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," *IETF RFC 7519*, May 2015. [Online]. Available: `https://www.rfc-editor.org/rfc/rfc7519`

[7] World Organisation for Animal Health (WOAH), "Joint WHO/WOAH Guidance on Stray Dog Population Management," *WOAH Terrestrial Animal Health Code*, Paris, France, 2022.

[8] A. M. Reese, M. C. Gates, and N. J. Kogan, "The Role of Social Media in Companion Animal Foster Care and Rescue Operations," *Frontiers in Veterinary Science*, vol. 9, Art. no. 842155, pp. 1–12, Mar. 2022, doi: 10.3389/fvets.2022.842155.

[9] S. L. J. Taylor and C. C. Phillips, "Citizen Science and Stray Animal Monitoring: Utilizing Mobile and Web GIS for Animal Welfare Reporting," *Journal of Applied Animal Welfare Science*, vol. 26, no. 3, pp. 312–327, 2023, doi: 10.1080/10888705.2021.1983842.

[10] M. J. Weiss, S. Gramann, and C. V. Spain, "Evaluation of Online Pet Adoption Portals on Length of Shelter Stay and Adoption Rates," *Animals*, vol. 11, no. 8, Art. no. 2351, Aug. 2021, doi: 10.3390/ani11082351.

[11] H. A. Signal, C. F. Taylor, and K. E. Stewart, "Community Dynamics and Crowdfunding Transparency in Animal Rescue Initiatives," *Human-Animal Interactions*, vol. 11, no. 1, pp. 45–59, Jan. 2023.

[12] S. Minaee, T. Mikolov, N. Nikzad, M. Chen, R. Socher, and J. Gao, "Large Language Models: A Survey," *arXiv preprint arXiv:2402.06196*, 2024.

[13] S. Karmakar, M. I. Joy, S. T. Imu, A. Biswas, J. A. Audity, and M. A. Rahman, "HyperID: Ontology-Grounded Explainable Breed-Ancestry Estimation for Mixed and Indigenous Companion Animals — A Neuro-Symbolic Approach," Dept. of CSE, Bangladesh University of Business and Technology, Dhaka, Bangladesh, 2024.

[14] C. R. Smith and D. M. L. Hughes, "Applications of Artificial Intelligence and Machine Learning in Veterinary Clinical Practice: Opportunities and Ethical Considerations," *Veterinary Clinics of North America: Small Animal Practice*, vol. 53, no. 4, pp. 789–803, Jul. 2023, doi: 10.1016/j.cvsm.2023.02.006.

[15] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, et al., "Learning Transferable Visual Models From Natural Language Supervision (CLIP)," in *Proc. 38th Int. Conf. Machine Learning (ICML)*, vol. 139, Jul. 2021, pp. 8748–8763.

[16] G. Gerganov, "GGML / GGUF: Large Model Inference on Commodity and Edge Hardware," *GitHub Repository*, 2023. [Online]. Available: `https://github.com/ggerganov/llama.cpp`

[17] ISO/IEC/IEEE, *Systems and Software Engineering — Requirements Engineering*, ISO/IEC/IEEE Standard 29148:2018, Nov. 2018, doi: 10.1109/IEEESTD.2018.8559686.

[18] International Organization for Standardization, *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*, ISO/IEC Standard 25010:2011, Mar. 2011.

[19] Open Web Application Security Project (OWASP), "OWASP Top 10: 2021 The Ten Most Critical Web Application Security Risks," *OWASP Foundation*, 2021. [Online]. Available: `https://owasp.org/Top10/`

[20] World Wide Web Consortium (W3C), "Web Content Accessibility Guidelines (WCAG) 2.1," *W3C Recommendation*, Jun. 2018. [Online]. Available: `https://www.w3.org/TR/WCAG21/`

[21] K. Beck et al., "Manifesto for Agile Software Development," *Agile Alliance*, 2001. [Online]. Available: `https://agilemanifesto.org/`